Session 7
Lab Report
Kotlin Tip Calculator (Jetpack Compose)

Name : Pawankumar
Course : Android App Development with Gen AI

Introduction

This laboratory session focuses on building a Tip Calculator using Kotlin and Jetpack Compose in Android Studio. The application allows users to enter the bill amount, select the tip percentage, and optionally round up the tip value. The project demonstrates the use of: • Kotlin functions • Conditional logic • Numeric operations • String formatting • Jetpack Compose UI components • State management using remember and mutableStateOf

Objectives

Objectives of the Experiment: 1. Understand Kotlin numeric calculations. 2. Learn how to implement conditional logic using if statements. 3. Practice building UI using Jetpack Compose. 4. Implement user input fields and switches. 5. Format currency output using NumberFormat.

Tasks

Task	Description
Task 1	Create a new Android Studio project using Kotlin and Jetpack Compose.
Task 2	Design the user interface using Compose components such as Column, Text Field
Task 3	Implement the tip calculation logic using a Kotlin function.
Task 4	Display the calculated tip amount and allow optional rounding of the value.

Algoritm :

Step 1: Start the application.

Step 2: Accept bill amount from the user.

Step 3: Accept tip percentage.

Step 4: Convert the input values to numeric format.

Step 5: Calculate tip using formula: Tip = (TipPercent / 100) * Amount

Step 6: If round up option is enabled, apply ceiling function.

Step 7: Format result using currency format.

Step 8: Display calculated tip amount.

Step 9: End.

Program Flow

Flow of the Program:

User Input (Bill Amount + Tip %)

Convert Input to Numbers

Calculate Tip

Check Round-Up Option

Format Currency

Display Tip Amount

Code Explanation

Code Explanation: MainActivity: The MainActivity class is the entry point of the Android application. It sets the Compose UI layout using setContent(). TipTimeLayout(): This composable function creates the entire UI layout including input fields, switch, and result display. EditNumberField(): Reusable composable used to create numeric input fields for bill amount and tip percentage.

RoundTheTipRow(): This composable displays the switch that allows the user to enable or disable rounding of the tip. calculateTip(): This function performs the tip calculation. It multiplies the amount by the tip percentage and optionally rounds the result using the ceil() function.

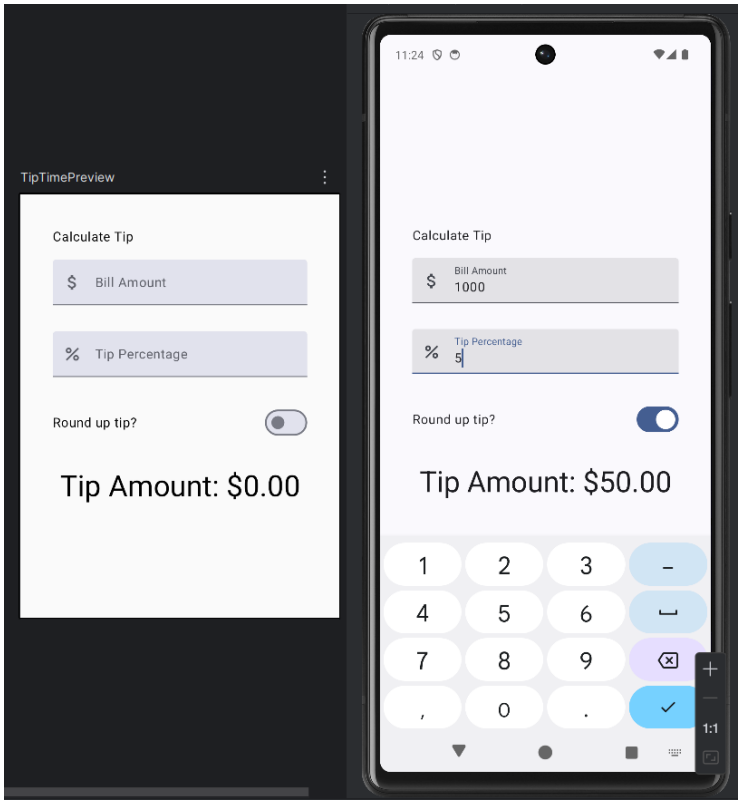
Kotlin Source Code

```
package com.example.tipcalculator

import android.os.Bundle
import androidx.activity.ComponentActivity import
androidx.activity.compose.setContent import
androidx.activity.enableEdgeToEdge import
androidx.annotation.DrawableRes import
androidx.annotation.StringRes
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll import
androidx.compose.material3.*
import androidx.compose.runtime.* import


androidx.compose.ui.Alignment import
androidx.compose.ui.Modifier
import androidx.compose.ui.res.painterResource import
androidx.compose.ui.res.stringResource import
androidx.compose.ui.text.input.ImeAction import
androidx.compose.ui.text.input.KeyboardType import
androidx.compose.ui.tooling.preview.Preview import
```

```
androidx.compose.ui.unit.dp
import com.example.tipcalculator.ui.theme.TipCalculatorTheme import
java.text.NumberFormat
import kotlin.math.ceil

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState) enableEdgeToEdge()

        setContent {
            TipCalculatorTheme { Surface(
                        modifier = Modifier.fillMaxSize(),
                        color = MaterialTheme.colorScheme.background
                    ) {
                        TipTimeLayout()
                    }
            }
        }
    }
}

@Composable
fun TipTimeLayout() {

    var amountInput by remember { mutableStateOf("") } var
    tipInput by remember { mutableStateOf("") } var roundUp
    by remember { mutableStateOf(false) }

    val amount = amountInput.toDoubleOrNull() ?: 0.0 val
    tipPercent = tipInput.toDoubleOrNull() ?: 0.0

    val tip = calculateTip(amount, tipPercent, roundUp)

    Column(
            modifier = Modifier
                .padding(40.dp)
            .verticalScroll(rememberScrollState()), horizontalAlignment =
        Alignment.CenterHorizontally, verticalArrangement =
        Arrangement.Center
    ) {

        Text(
            text = stringResource(R.string.calculate_tip), modifier =
            Modifier
                .padding(bottom = 16.dp)
                .align(Alignment.Start)
        )

        EditNumberField(
            label = R.string.bill_amount,
            leadingIcon = R.drawable.money,
            keyboardOptions = KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Number,
                imeAction = ImeAction.Next
            ),
            value = amountInput,
            onValueChanged = { amountInput = it },
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 32.dp)
        )

        EditNumberField(
            label = R.string.how_was_the_service, leadingIcon =
            R.drawable.percent, keyboardOptions =
            KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Number,
                imeAction = ImeAction.Done
            ),
```

```
                value = tipInput,
                onValueChanged = { tipInput = it }, modifier
                = Modifier
                    .fillMaxWidth()
                    .padding(bottom = 32.dp)
            )

            RoundTheTipRow(
                roundUp = roundUp,
                onRoundUpChanged = { roundUp = it }, modifier =
                Modifier.padding(bottom = 32.dp)
            )

            Text(
                text = stringResource(R.string.tip_amount, tip), style =
                MaterialTheme.typography.displaySmall
            )
        }
    }

    private fun calculateTip( amount:
        Double, tipPercent: Double,
        roundUp: Boolean
    ): String {

        var tip = tipPercent / 100 * amount

        if (roundUp) {
            tip = ceil(tip)
        }

        return NumberFormat.getCurrencyInstance().format(tip)
    }
```

Output :



Conclusion

Conclusion: The Kotlin Tip Calculator successfully demonstrates the use of Jetpack Compose UI components and Kotlin programming concepts. The application allows users to calculate tips easily while learning about numeric operations, state management, and conditional logic. This lab helps students strengthen their understanding of modern Android development practices using Kotlin.